C Important Questions

1. Explain the importance of C.
Ans:

- **Middle Level Language:** Combines the features of high level and low level languages system (like assembly language) and application programming C like high level language.
- **Structured Programming Language:** Encourages the use of functions, loops, decision making statements and modular programming makes complex programs easier to understand, debug and maintain.
- **Rich Set of Operations and Data Types:** Provides a wide variety of operations (arithmetic, logical, bitwise, etc) and data types enabling efficient programming allows the development of compact and faster programs.
- **Portablity:** Programs written in C can be compiled and run on different machines with little or no modification this makes C ideal for writing system software and portable applications.

2. Basic structure of C.
Ans:

- **Documentation Section:**
  - Purpose of the program, author, date and any additional informations.
  - Not executed by the compilers.
  - Ex: /* Program by Ashwith Frank*/
- **Link Section:**
  - Used to include header file and libraries.
  - Tells the compiler to link functions from the standard libraries.
  - Ex: #include <stdio.h>
- **Defination Section:**
  - Defines constants and macros.
  - Typically uses #define
  - Ex: #define PI 3.14
- **Global Declaration Section:**
  - Declares global variables and function prototypes.
  - These are accessible to all the functions in the program.

© 2025 AlphaOmega Studio • Ashwith Frank

- Ex: int total;
  void display();

- **main() Function Section:**
  - Every C program must have a main() function.
  - It contains two main parts:
    - Declaration Part.
    - Executable Part.
  - Ex: #include <stdio.h>
    void main()
    {
    int a, b, sum;
    a = 10;
    b = 20;
    sum = a + b;
    printf(“Sum of %d and %d = %d”, a, b, sum);
    }
- **Subprogram Section:**
  - It is also called as user defined functions’ section.
  - Contains user defined functions written outside the main() function.
  - Helps in modular programming and code reuse.
  - Ex: void display()
    {
    printf(“Hello World”);
    }

3. Explain #include and #define.

Ans:

- **#define**
  - Used to define constants or macros.
  - Replaces values or code before compilation.
  - No memory is allocated for defined constants.
  - Makes code easier to modify and read.
  - Ex: #define PI 3.14
- **#include**
  - Used to include header files in a C program.

© 2025 AlphaOmega Studio • Ashwith Frank

- Brings predefined functions and declarations into the program.
- Helps reuse standard libraries (like input/output, math).
- Processed before compilation (preprocessor directive).
- Ex: #include <stdio.h>

4. What are trigraph characters?
Ans: Trigraph characters are special sequences in C used to represent characters that are not available on some keyboards.
Ex: ??= → #
??/ → \
??' → ^

5. Explain C tokens.
Ans: C tokens are the smallest individual units of a C program that are meaningful to the compiler.
Types of C tokens:
- Keywords
- Identifiers
- Constants
- Strings
- Special Symbols
- Operators

Ex: int, sum, =, 10, +, 20, ;

6. Explain data types and variables.
Ans:
- **Data Types:** Data types specify the type of data a variable can store and the amount of memory allocated.
  Types of data types:
  - Primary (Basic) Data Types:
    - int – stores whole numbers
    - float – stores decimal numbers
    - char – stores single character
    - double – stores large decimal numbers
  - Derived Data Types:
    - Formed using basic data types

© 2025 AlphaOmega Studio • Ashwith Frank

        - Examples: array, pointer, structure, union
    - User-Defined Data Types:
        - Created by the programmer
        - Examples: struct, union, enum, typedef
    - Void Data Type:
        - Represents no value
        - Used in functions that return nothing (void)
- **Variables:** A variable is a named memory location used to store data values during program execution.
  Ex: int age = 20;
  float marks = 85.5;
  char grade = 'A';

7. What overflow and underflow of data?
Ans:

- **Overflow:** Overflow happens when a value exceeds the maximum limit of a data type.
    - Value becomes incorrect or wraps around
    - Occurs due to limited memory size
    - Common in integer and floating-point types

  Ex: char a = 127;
  a = a + 1; // Overflow
- **Underflow:** Underflow happens when a value goes below the minimum limit of a data type.
    - Value becomes incorrect or wraps around
    - Occurs when subtracting beyond minimum limit
    - Common in signed data types

  Ex: char b = -128;
  b = b - 1; // Underflow

8. What is operator? Explain its types.
Ans: An operator is a symbol that performs a specific operation on operands (variables or values).

- **Arithmetic Operators:** Used for mathematical calculations. Ex: + - * / %
- **Relational Operators:** Used to compare two values. Ex: < > <= >= == !=

© 2025 AlphaOmega Studio • Ashwith Frank

- **Logical Operators:** Used to combine conditions. Ex: && || !
- **Assignment Operators:** Used to assign values to variables. Ex: = += -= *= /=
- **Increment and Decrement Operators:** Used to increase or decrease value by 1. Ex: ++ –
- **Bitwise Operators:** Used for bit-level operations. Ex: & | ^ ~ << >>
- **Conditional Operator:** A shorthand for if-else. Ex: ?:
- **Special Operators:** Used for special purposes. Ex: sizeof, ,, &, *

9. Explain type conversion.
Ans: Type conversion is the process of converting one data type into another during program execution.

- **Implicit Type Conversion (Automatic):**
  - Done automatically by the compiler.
  - Converts smaller data type to larger data type.
  - No data loss in most cases.
  - Ex: int a = 10;
    float b = a; // int to float
- **Explicit Type Conversion (Type Casting):**
  - Done by the programmer.
  - Uses casting operator (type).
  - May cause data loss.
  - Ex: float x = 5.7;
    int y = (int)x; // float to int

10. Explain input and output functions.
Ans: Input and output functions are used to take data from the user and display output on the screen. All input/output functions are defined in the stdio.h header file.

- **Input Functions:**
  - scanf()
    - Used to read input from the keyboard.
    - Requires format specifiers.
    - Stores input in variables.
    - Ex: int a;
      scanf("%d", &a);

© 2025 AlphaOmega Studio • Ashwith Frank

    - getchar()
        - Reads a single character.
        - No format specifier needed.
        - Ex: char ch;
              ch = getchar();
- **Output Functions:**
    - printf()
        - Used to display output on the screen.
        - Uses format specifiers.
        - Ex: printf("Value = %d", a);
    - putchar()
        - Prints a single character.
        - Ex: putchar(ch);

11. Explain decision making.
Ans: Decision making allows a program to choose different paths of execution based on given conditions. It is mainly used to control the flow of a program.

- If Statement.
    - Simple if
    - If - else
    - Nested if
    - If - else Ladder
- Switch (expression).
- Conditional Operator.
- Goto Statement.

(Refer notes for more 🙂🙏)

12. Explain looping statements.
Ans: Looping statements are used to execute a block of code repeatedly until a given condition becomes false. They help reduce code repetition and make programs efficient.

- **for loop:**
    - Used when the number of iterations is known.
    - Initialization, condition, and increment/decrement in one line.
    - Ex: for (int i = 1; i <= 5; i++)
              printf("%d ", i);

© 2025 AlphaOmega Studio • Ashwith Frank

- **while loop:**
  - Condition is checked before loop execution.
  - Used when number of iterations is not fixed.
  - Ex: int i = 1;
    while (i <= 5)
    {
    printf("%d ", i);
    i++;
    }
- **do-while loop:**
  - Executes loop body at least once.
  - Condition is checked after execution.
  - Ex: int i = 1;
    do
    {
    printf("%d ", i);
    i++;
    } while (i <= 5);

13. Explain break and continue statements.
Ans:

- **Break Statement:** The break statement is used to terminate a loop or switch statement immediately.
  - Exits from the loop or switch.
  - Control moves to the statement after the loop.
  - Used to stop execution when a condition is met.
  - Ex: for (int i = 1; i <= 10; i++)
    {
    if (i == 5)
    break;
    printf("%d ", i);
    }
- **Continue Statement:** The continue statement is used to skip the current iteration of a loop.
  - Skips remaining statements in the loop body.
  - Moves control to next iteration.
  - Loop does not terminate.

© 2025 AlphaOmega Studio • Ashwith Frank

- Ex: for (int i = 1; i <= 5; i++)
  {
  if (i == 3)
  continue;
  printf("%d ", i);
  }

14. (ERROR INV_QUE)
Ans: (ERROR INV_ANS)

15. What is an array? Explain its types.
Ans: An array is a collection of elements of the same data type stored in contiguous memory locations and accessed using a single name with an index.

- **One-Dimensional Array (1D):**
  - Stores elements in a single row.
  - Uses one index.
  - Ex: int a[5] = {10, 20, 30, 40, 50};
- **Two-Dimensional Array (2D):**
  - Stores elements in rows and columns (matrix form).
  - Uses two indices.
  - Ex: int a[2][2] = {{1, 2}, {3, 4}};
- **Multi-Dimensional Array:**
  - Array with more than two dimensions.
  - Used for complex data representation.
  - Ex: int a[2][2][2];

16. How to initialize arrays in runtime and compile time?
Ans:

- **Compile Time Initialization:** Array values are assigned at the time of declaration.
  - Values are fixed in the program.
  - Done before program execution.
  - Simple and fast.
  - Ex: int a[5] = {10, 20, 30, 40, 50};
- **Run Time Initialization:** Array values are assigned during program execution (usually using input).

© 2025 AlphaOmega Studio • Ashwith Frank

- Values are entered by the user.
- Flexible and dynamic.
- Done using loops.
- Ex: int a[5];

```
for (int i = 0; i < 5; i++)
{
    scanf("%d", &a[i]);
}
```

17. What is a string? Explain 5 string handling functions.
Ans: A string is a sequence of characters stored in a character array and terminated by a null character ('\0').
Ex: char name[10] = "Ash";

- **String Handling Functions:** String handling functions are defined in the string.h header file.
  - strlen()
    - Returns the length of a string (excluding '\0').
    - Ex: strlen(name);
  - strcpy()
    - Copies one string into another.
    - Ex: strcpy(dest, src);
  - strcat()
    - Appends one string to another.
    - Ex: strcat(str1, str2);
  - strcmp()
    - Compares two strings.
    - Returns 0 if both are equal.
    - Ex: strcmp(str1, str2);

(Refer notes for more 🙂🙏)

18. Explain 5 character handling functions.
Ans: Character handling functions are used to test and manipulate individual characters. They are defined in the ctype.h header file.

- isalpha()
  - Checks whether a character is an alphabet (A–Z or a–z).
  - Ex: isalpha('A'); // true

© 2025 AlphaOmega Studio • Ashwith Frank

- isdigit()
    - Checks whether a character is a digit (0–9).
    - Ex: isdigit('5'); // true
- islower()
    - Checks whether a character is in lowercase.
    - Ex: islower('a'); // true
- toupper()
    - Converts a lowercase character to uppercase.
    - Ex: toupper('a'); // 'A’

(Refer notes for more 🙂🙏)

19. What are user defined functions? Explain its needs and categories.
Ans: A user-defined function is a function created by the programmer to perform a specific task. It helps divide a program into smaller, manageable modules.

- **Needs of User-Defined Functions:**
    - Reusability: A function can be called multiple times, avoiding repetitive code.
    - Modularity: Breaks a large program into smaller, understandable parts.
    - Ease of Maintenance: Errors are easier to locate and fix in smaller modules.
    - Readability: Programs are easier to read and understand.
    - Debugging: Testing individual functions is easier than testing a full program.
- **Categories of User-Defined Functions:**
    - With Arguments and With Return Value:
        - Accepts input and returns output.
        - Ex:
```
int add(int a, int b)
{
    return a + b;
}
```
    - With Arguments and No Return Value:
        - Accepts input but does not return anything.
        - Ex:
```
void display(int a) {
    printf("%d", a);
}
```

© 2025 AlphaOmega Studio • Ashwith Frank

- No Arguments and With Return Value:
    - Does not take input but returns output.
    - Ex: int getValue() {
        return 100;
    }
- No Arguments and No Return Value:
    - Does not take input and does not return anything.
    - Ex: void greet() {
        printf("Hello!");
    }

20. Explain functions' elements.
Ans: A function is made up of several important elements that define what it does, what it accepts, and what it returns.

- **Function Name:**
    - The name given to a function to identify it.
    - Should be meaningful and follow identifier rules.
    - Ex: int add(int a, int b) // 'add' is the function name
- **Return Type:**
    - Specifies the type of value the function will return.
    - Can be int, float, char, void, etc.
    - If the function doesn't return any value, void is used.
    - Ex: int add(int a, int b) // return type is int
        void display() // return type is void
- **Parameters (Arguments):**
    - Inputs passed to the function.
    - Allow functions to work with different values.
    - Can be zero or more parameters.
    - Ex: int add(int a, int b) // 'a' and 'b' are parameters
- **Function Body:**
    - Contains statements that define the task
    - Enclosed in { }
    - Ex: int add(int a, int b) {
        return a + b; // function body
    }

© 2025 AlphaOmega Studio • Ashwith Frank

21. Explain actual and formal parameters.
Ans:

- **Actual Parameters:**
  - Also called arguments.
  - Values passed to a function when it is called.
  - Exist in the calling function (like main).
- **Formal Parameters:**
  - Variables declared in the function definition
  - Receive the values of actual parameters
  - Exist in the called function

22. What is structure and union? Differentiate.
Ans:

- **Structure:**
  - A structure is a user-defined data type that groups different types of variables under a single name.
  - Each member has its own memory.
  - Used when you want to store related data together.
- **Union:**
  - A union is a user-defined data type that groups different types of variables but shares the same memory among all members.
  - Only one member can store value at a time.
  - Saves memory compared to structures.

23. (**ERROR INV_QUE)**
Ans: (ERROR INV_ANS)

24. Explain structure within structures, Structure's functions.
Ans:

- **Structure within Structures (Nested Structure):**
  - A nested structure is when a structure contains another structure as a member.
  - It helps to organize complex data logically.
- **Structure Functions (Operations on Structures):**
  - C does not have built-in functions for structures, but you can perform operations using functions:

© 2025 AlphaOmega Studio • Ashwith Frank

- a) Passing Structure to Function:
    - Pass structure as value or reference (pointer).
- b) Returning Structure from Function:
    - Functions can return a structure.
- c) Modifying Structure Using Pointer:
    - Use pointers to modify structure members inside function.

25. What are the pointers? How to initialize? Call by reference and call by value. Advantages.
Ans: A pointer is a variable that stores the memory address of another variable.

- **How to Initialize a Pointer:**
    - Method 1: At declaration:
      int a = 5;
      int *p = &a;
    - Method 2: Later assignment:
      int a = 10;
      int *p;
      p = &a;

- **Call by Value vs Call by Reference:**
    - a) Call by Value:
        - Copies the value of actual parameter to formal parameter.
        - Changes in function do not affect original variable.
    - b) Call by Reference:
        - Pass address of variable using pointer.
        - Changes in function affect original variable.

- **Advantages of Pointers:**
    - Memory Efficiency: Avoids copying large data
    - Call by Reference: Functions can modify original data
    - Dynamic Memory Allocation: Enables use of heap memory
    - Efficient Arrays and Strings Handling: Access elements faster

© 2025 AlphaOmega Studio • Ashwith Frank

- Flexibility: Can create complex structures like linked lists, trees

26. Explain file handling functions. How to create a file?
Ans: (Skip or Refer the notes 🙂🙏)

"Preparing this PDF wasn’t easy – it took a lot of time, effort, and brainpower. I went through the questions, picked the most important ones, and made it as simple and useful as possible for you all. Even though it was challenging, I did it for you, hoping it makes your exam a little easier. Give it your best and make it count, you’ve got this!” – Ashwith Frank

© 2025 AlphaOmega Studio • Ashwith Frank